

Snap7 进阶功能指南 —— 未覆盖但实用的 API

导读

5 个入门项目覆盖了日常 90% 的通讯需求。本文补充 4 个"用得着但没在项目中演示"的常用功能。每个都附带可直接集成到现有项目的代码示例。

1. ReadMultiVars / WriteMultiVars —— 批量读写

什么时候用？

项目 5 的轮询是**逐个读取**——10 个地址 = 10 次 TCP 往返，每次 10-20ms，总共 100-200ms。用 ReadMultiVars 打包成一个请求：10 个地址 = **1 次** TCP 往返，约 15-30ms。差距随地址数量线性放大。

数据结构和函数

```
// snap7.h 中的定义
typedef struct {
    int    Area;           // S7AreaPE / S7AreaPA / S7AreaMK / S7AreaDB
    int    WordLen;        // S7WLBit / S7WLByte / S7WLWord / S7WLDWord / S7WLReal
    int    Result;         // 输出：每个 item 的返回码（0=成功）
    int    DBNumber;       // DB 号（仅 Area=DB 时有效，否则填 0）
    int    Start;          // 起始字节偏移
    int    Amount;         // 读取数量（单位由 WordLen 决定）
    void   *pdata;         // 存放结果的缓冲区（调用前分配好）
} TS7DataItem;

// C API
int Cli_ReadMultiVars(S7Object Client, PS7DataItem Items, int ItemsCount);

// C++ 类
int TS7Client::ReadMultiVars(PS7DataItem Items, int ItemsCount);
```

WordLen 含义

WordLen 常量	值	每个"Amount"的大小	对应数据类型
S7WLBit	0x01	1 bit	BOOL
S7WLByte	0x02	1 byte	BYTE / INT8 / UINT8
S7WLWord	0x04	2 bytes	INT16 / UINT16
S7WLDWord	0x06	4 bytes	INT32 / FLOAT
S7WLReal	0x08	4 bytes	FLOAT（同上，历史原因）

完整示例：一次读 3 个不同地址

```

#include <QByteArray>
#include "snap7.h"

bool readBatchExample(TS7Client &client)
{
    // ---- 准备 3 个 item ----
    TS7DataItem items[3];

    // Item 0: 读 DB1 的 DBD4 (4个字节, FLOAT)
    items[0].Area    = S7AreaDB;
    items[0].WordLen = S7WLByte;
    items[0].DBNumber = 1;
    items[0].Start    = 4;
    items[0].Amount   = 4;
    QByteArray buf0(4, '\0');
    items[0].pdata    = buf0.data();

    // Item 1: 读 M10.0 开始的 2 个字节 (INT16)
    items[1].Area    = S7AreaMK;
    items[1].WordLen = S7WLByte;
    items[1].DBNumber = 0;
    items[1].Start    = 10;
    items[1].Amount   = 2;
    QByteArray buf1(2, '\0');
    items[1].pdata    = buf1.data();

    // Item 2: 读 I0.0 开始的 1 个字节
    items[2].Area    = S7AreaPE;
    items[2].WordLen = S7WLByte;
    items[2].DBNumber = 0;
    items[2].Start    = 0;
    items[2].Amount   = 1;
    QByteArray buf2(1, '\0');
    items[2].pdata    = buf2.data();

    // ---- 一次请求全部读完 ----
    int result = client.ReadMultiVars(items, 3);
    if (result != 0)
        return false;

    // ---- 检查每个 item 的独立结果 ----
    if (items[0].Result == 0) {
        // buf0 里有 4 字节, 按大端解析 FLOAT
        quint32 bits = ((quint8)buf0[0] << 24) | ((quint8)buf0[1] << 16)
                     | ((quint8)buf0[2] << 8)  | (quint8)buf0[3];

        float f;
    }
}

```

```

        std::memcpy(&f, &bits, 4);
        // f = 读到的 FLOAT 值
    }

    if (items[1].Result == 0) {
        // buf1 里有 2 字节，按大端解析 INT16
        quint16 v = (quint16)((quint8)buf1[0] << 8 | (quint8)buf1[1]);
    }

    return true;
}

```

集成到项目 5 的 pollAllItems() 中

将原来的 for 循环替换为：

```

void PlcWorker::pollAllItems()
{
    if (!m_client->Connected()) { /* 重连逻辑... */ return; }

    // 准备 ReadMultiVars 需要的数组
    QVector<TS7DataItem> items(m_items.size());
    QVector<QByteArray> buffers(m_items.size());

    for (int i = 0; i < m_items.size(); ++i) {
        const MonitorItem &mi = m_items[i];
        int size = sizeForType(mi.valType);
        buffers[i].resize(size);

        items[i].Area      = mi.area == 3 ? S7AreaDB :
                               mi.area == 0 ? S7AreaMK :
                               mi.area == 1 ? S7AreaPE : S7AreaPA;
        items[i].WordLen   = S7WLByte;
        items[i].DBNumber  = mi.dbNum;
        items[i].Start     = mi.start;
        items[i].Amount    = size;
        items[i].pdata     = buffers[i].data();
    }

    int result = m_client->ReadMultiVars(items.data(), items.size());
    if (result != 0) return;

    for (int i = 0; i < m_items.size(); ++i) {
        if (items[i].Result == 0)
            emit pollingData(i, m_items[i].area, m_items[i].dbNum,
                             m_items[i].start, buffers[i]);
    }
}

```

```
        else
            emit pollingData(i, 0, 0, 0, QByteArray()); // 失败
    }

    emit pollingTick();
}
```

2. SZL（系统状态列表）—— 读 PLC 诊断信息

SZL（System Zustand Liste，系统状态列表）是西门子 PLC 的诊断数据。通过它你可以读取：

- CPU 模块信息（型号、固件版本）
- 机架上的模块列表
- LED 指示灯状态
- 诊断缓冲区（故障记录）

函数

```
// 读取指定 ID/Index 的 SZL
int Cli_ReadSZL(S7Object Client, int ID, int Index,
                TS7SZL *pUsrData, int *Size);
int Cli_ReadSZLList(S7Object Client, TS7SZLList *pUsrData, int *ItemsCount);
```

常用 SZL ID

ID (十六进制)	含义	例子
0x0011	CPU 模块识别数据	订货号、固件版本
0x001C	诊断缓冲区	故障记录、事件时间戳
0x0111	机架/槽位状态	每个槽装了什么模块
0x0131	模块 LED 状态	SF/BF/MAINT 灯状态

示例：读 CPU 模块信息

```
void readCpuIdentification(TS7Client &client)
{
    TS7SZL szl;
    int size = sizeof(TS7SZL);

    // ID=0x0011, Index=0x0000 = CPU 模块识别
    int result = client.ReadSZL(0x0011, 0x0000, &szl, &size);
    if (result != 0) return;

    // szl.Data 里是原始数据，结构复杂，这里只展示关键字段
}
```

```
// 前 2 字是 Header (N_DR = 记录数)
// 后面是各记录，每记录格式：
//   Index(2字节) + 订货号(20字节ASCII) + 版本(2字节)

// 获取订货号（从 Data 偏移 2 开始的 20 字节）
char orderCode[21];
std::memcpy(orderCode, szl.Data + 2, 20);
orderCode[20] = '\0';

qDebug() << "CPU 订货号:" << orderCode;
}
```

使用场景

场景	读取内容
上位机启动时	读 CPU 模块信息，确认连对了设备
故障排查	读诊断缓冲区，查看历史故障
机架扫描	读槽位状态，发现所有模块

3. 安全 / 密码 —— 访问受保护的 PLC

S7-1200/1500 可以设置密码保护，限制上位机访问级别。snap7 提供了会话密码功能。

函数

```
int Cli_SetSessionPassword(S7Object Client, char *Password);
int Cli_ClearSessionPassword(S7Object Client);
int Cli_GetProtection(S7Object Client, TS7Protection *pUsrData);
```

连接带密码的 PLC

```
void connectWithPassword(TS7Client &client, const char *ip, const char *pwd)
{
    // 1. 先连接
    int result = client.ConnectTo(ip, 0, 1);
    if (result != 0) return;

    // 2. 设置会话密码
    result = client.SetSessionPassword((char *)pwd);
    if (result != 0) {
        qDebug() << "密码错误或不需要密码";
    }
}
```

```
// 3. 现在可以正常读写
// ...

// 4. 断开前清除密码
client.ClearSessionPassword();
}
```

读取 PLC 保护级别

```
TS7Protection prot;
int result = client.GetProtection(&prot);
// prot.sch_schal — 保护级别（1-4）
// 1 = 无保护
// 2 = 写保护
// 3 = 读/写保护
// 4 = 完全保护（需要密码）
```

注意事项

- `SetSessionPassword` 必须在连接成功后、任何读写操作前调用
- 密码是明文传输的，不要在不可信网络上使用
- S7-1200 v4.0+ 支持更强的密码保护，snap7 的兼容性有限
- TIA Portal 的"PLC 访问保护"和"HMI 访问"是两个独立设置

4. 异步 API —— snap7 内建的异步机制

snap7 提供了一套基于回调的异步 API，所有 `As*` 前缀的函数都是异步版本。

机制

```
Cli_AsReadArea(...) // 发起异步读，立即返回
|
▼ (内部线程处理)
|
Cli_CheckAsCompletion() // 轮询是否完成
Cli_WaitAsCompletion() // 阻塞等待完成
Cli_SetAsCallback() // 或设置回调函数，完成时自动调用
```

对比：什么时候用哪个

方式	适用场景	优点	缺点
QThread (项目做法)	通用	完全控制，跟 Qt 信号槽完美配合	需要理解多线程

snap7 异步 API	无 Qt / 纯 C 项目	不需要外部线程库	需要轮询或用自己线程
--------------	---------------	----------	------------

对于 Qt 项目，推荐继续用 QThread 方案，它和 Qt 框架集成更好。但如果你的项目不用 Qt，snap7 的异步 API 是内置的选择。

总结：API 速查表

功能分类	函数	5 个项目是否覆盖	本文是否覆盖
连接管理	ConnectTo / Disconnect	✓	
连接状态	Connected / PlcStatus	✓	
连接参数	SetParam / SetConnectionType		
读单地址	DBRead / MBRead / EBRead / ABRead	✓	
写单地址	DBWrite / MBWrite / EBWrite / ABWrite	✓	
批量读写	ReadMultiVars / WriteMultiVars		✓
读时间	GetPlcDateTime / SetPlcDateTime		
诊断信息	ReadSZL / ReadSZLList		✓
密码/安全	SetSessionPassword / GetProtection		✓
块操作	Upload / Download / DBGet / DBFill		
控制命令	PlcStop / PlcHotStart / CopyRamToRom		
异步 API	AsReadArea 等		✓
服务器模式	Srv_* 系列函数		
伙伴模式	Par_* 系列函数		